

# Predictive Maintenance using SAP and AWS IoT to reduce operational cost

by Patrick Leung | on 06 MAY 2021 | in [AWS IoT Analytics](#), [Internet of Things](#), [SAP on AWS](#), [Technical How-to](#) | [Permalink](#) |

[↪ Share](#)

*This post was written by Kenny Rajan, Patrick Leung, Scott Francis, Will Charlton & Ganesh Suryanarayan.*

The convergence of Operational Technology (OT) and Information Technology (IT) is reinventing the way companies drive manufacturing efficiency. From the shop floor at the Programmable Logic Controller (PLC) level to the Manufacturing Execution System (MES), all the way up to [SAP Plant Maintenance \(PM\)](#) or [S/4HANA Asset Management](#).

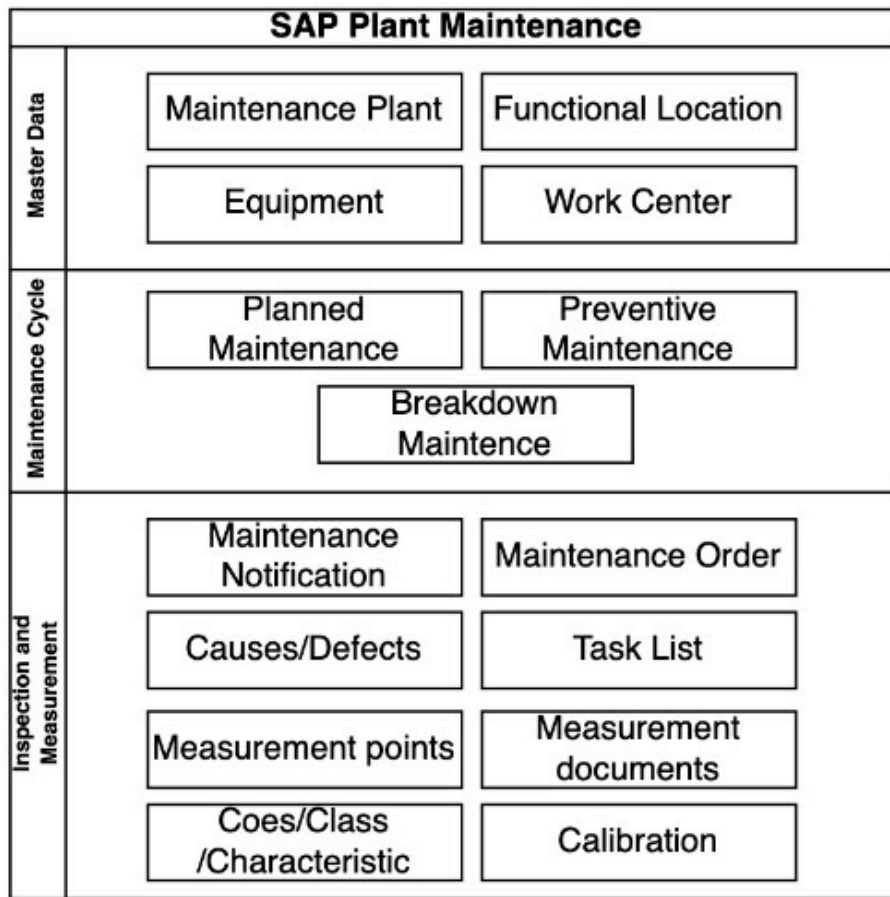
With [AWS IoT solutions](#), industrial companies can digitize processes, transform business models, and improve performance and productivity, while decreasing waste. These are some examples of how customers can apply AWS IoT solutions to improve the performance and productivity of industrial processes.

- [Use AWS IoT to build predictive quality models in industrial operations](#): See how AWS IoT can collect and analyze data from industrial operations to build predictive quality models. [Learn more](#)
- [Use AWS IoT to support predictive maintenance in industrial operations](#): See how AWS IoT can help plan preventive maintenance to reduce unplanned downtime. [Learn more](#)

## Introduction

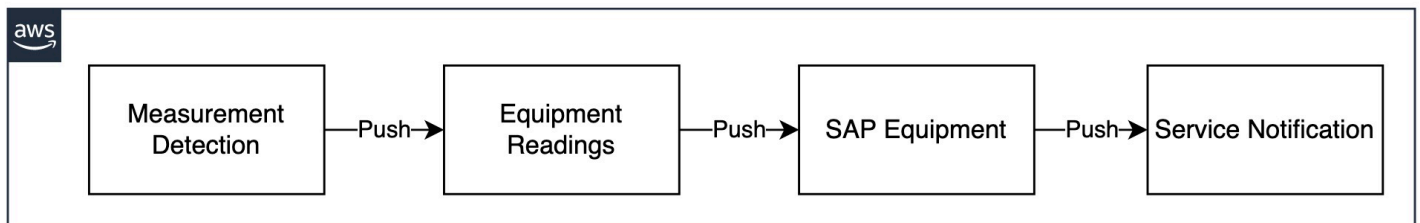
In this blog post, we show how to integrate AWS IoT solutions with SAP for predictive maintenance. SAP Plant Maintenance (PM) comprises a set of data and processes to maintain the high availability of technical systems. This helps customers move away from outdated interval or breakdown-based maintenance cycles to a planned or preventative maintenance cycle.

The following diagram illustrates the business processes that effectively plan, inspect, record and act on information about various equipment's, devices and assets within an organization in a timely manner. More detailed information about each of the technical objects can be found in [SAP documentation](#).



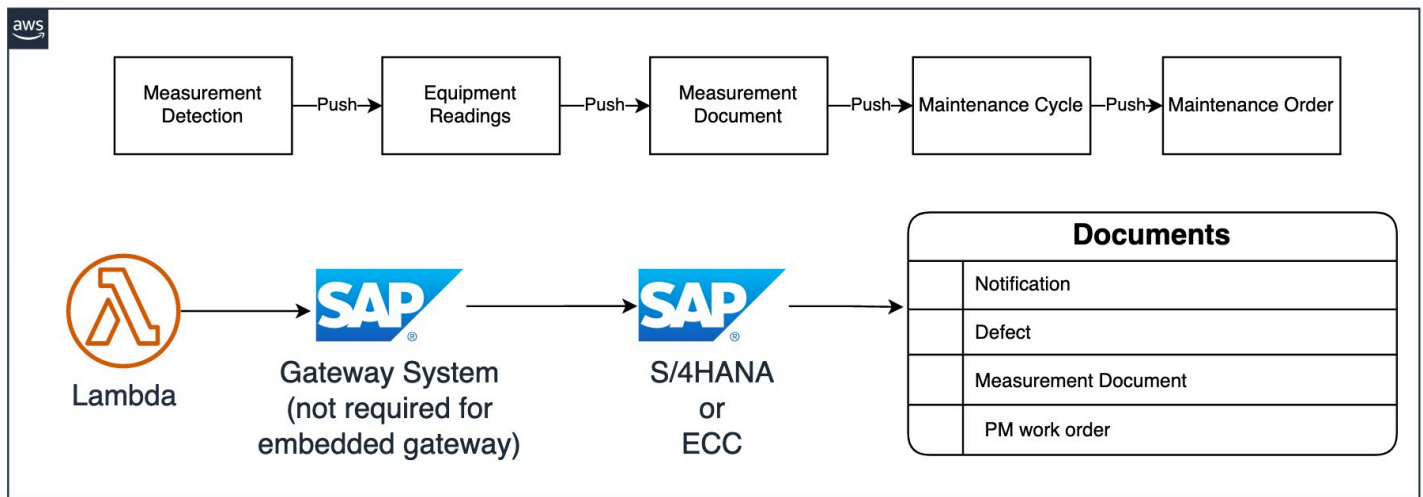
#### Example Business Process 1:

A lot of equipment readings are recorded in plant maintenance and customer service processes. In case of a potential disruptive event in an assembly or sub-component, a service notification can be triggered.



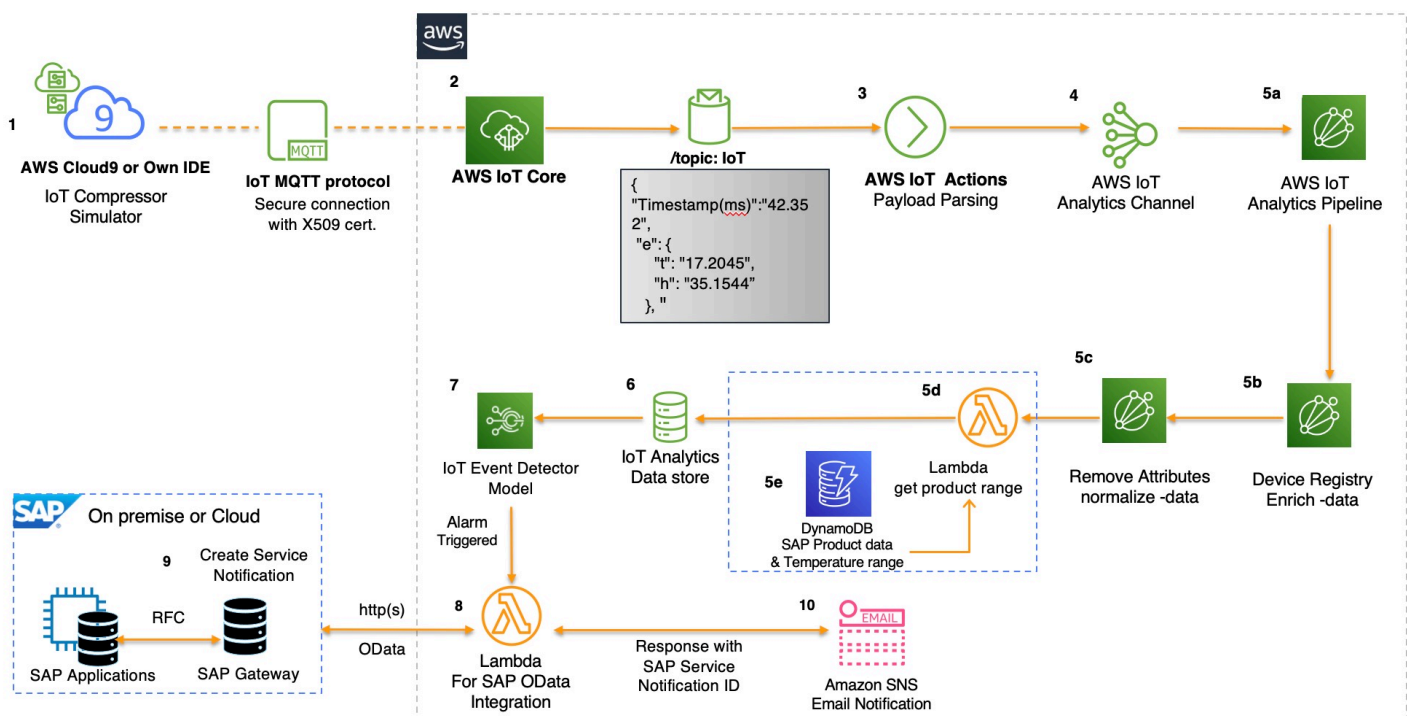
#### Example Business Process 2:

Alternatively, a maintenance plan can be set up to trigger a maintenance order based on thresholds. Once the device at the edge detects an anomaly, an [AWS Lambda function](#) can be triggered to create a measurement document in SAP for the assembly or sub assembly. This would in turn trigger the creation of a maintenance order.



## Architecture and Solution Overview

To keep it simple, we are using an IoT device simulator to mimic an equipment failure. Then, we show how to use AWS IoT services for early failure detection. For the integration back into SAP, we leverage [REST-based OData \(Open Data Protocol\) services](#) and AWS Lambda to create service notification for maintenance activities. SAP maintenance notification plays an important role in plant maintenance operations by proactively notifying an abnormal or exceptional situation. The solution consists of the following components:



- 1. IoT Device Simulation environment:** Use [AWS Cloud9](#) or your own Integrated Development Environment (IDE) to simulate a compressor's temperature reading to [AWS IoT Core](#).
- 2. Security and Connectivity:** [AWS Cloud Development Kit \(CDK\)](#) deploys AWS IoT Core to register the [IoT thing](#) and generate [X.509 certificate](#). The X.509 certificate is installed on your simulation environment to establish connectivity to AWS IoT core. The simulated data is then sent to AWS IoT Core using Message Queuing Telemetry Transport (MQTT) protocol. In addition, AWS IoT provides a registry that helps you manage IoT things. In this example, we have a compressor maintained in this registry.

3. **Message payload parsing:** [AWS IoT action rules](#) listen for specific MQTT payload from the simulator, formats the data and send the data to [AWS IoT Analytics](#) services.
4. **Monitor messages in real time with AWS IoT Analytics:** AWS IoT Analytics channel receives the data from the AWS IoT Core. AWS IoT Analytics validates that the data is within specific threshold. Here, we setup the set ID, source, and data retention period and routes the channel to appropriate pipelines.
5. **Configure AWS IoT Analytics pipeline activities:**
  - a. **Process data:** Transform message attributes, filter entire messages. Here, you chain the activities together to process and prepare messages before storing them.
  - b. **Enrich data:** Enrich the data from the IoT core before sending the data to the AWS IoT Analytics data store. An activity that adds data from the AWS IoT device registry to your message.
  - c. **Normalize data:** Remove attributes from a message to normalize the IoT payload message to filter the required data at the root level.
  - d. **Transform data with AWS Lambda:** Call Lambda functions to transform and enhance the upstream message to get the product range from Amazon DynamoDB. Through this activity, AWS IoT Analytics can understand the standard temperature range for a given device by calling Amazon DynamoDB to determine if there is a temperature fluctuation in the data sent from the IoT simulator. In this example, this is the compressor.
  - e. **Retrieve SAP product data:** DynamoDB setup includes the SAP attributes, which are mapped against the device name. Using these attributes, IoT Analytics retrieves the SAP product information for further downstream processing.
6. **Data Store:** AWS IoT Analytics data store receives and stores your messages. Here, we configure a SQL query to select the most recent temperature reading over the period. Data sets are stored in [Amazon Simple Storage Service \(S3\)](#) and concurrently sent through to AWS [IoT Events](#) to capture the temperature status.
7. **Anomaly Detection:** Using AWS IoT Events Detector model to define the conditional (Boolean) logic that evaluates the incoming inputs to detect an anomaly event (temperature out of normal range for more than 15 mins). When an event is detected, it changes the state and triggers additional actions to IoT events. For this demonstration, this model is configured to call the SAP to create an SAP service notification using AWS Lambda.
8. **API integration to SAP Lambda OData:** This sample package contains a Lambda layer to connect with SAP and consume the OData services through REST API calls. When an anomaly is detected, IoT Event detector model invoke an AWS Lambda function and passes the SAP equipment number and functional location to create a Service notification in SAP.
9. **Create Service Notification:** HTTP POST request is initiated with the above payload to call the SAP OData service using the environment variables (Entity-Set Name, Service-name, SAP-host, SAP-port and SAP authentication). The HTTP POST response return the Service Notification number from the SAP system.
10. **Alert Message:** Lambda receive a response with the SAP service notification number. This is parsed and sent to an AWS Simple Notification Service (SNS) Topic to alert the subscribers.

## Solution Deployment

There are two key steps in the deployment process. The first part is to configure the SAP system and the second part is the AWS deployment. The detailed steps are shown below.

### SAP Configuration

In the SAP backend system, create an OData service. There are two possible ways to achieve this.

1. Activate the OData Service using **SAP EAM\_NTF\_CREATE** SAP OData service.
2. Create an SAP OData service using a customized structure for the payload

For this deployment, we are using a customized structure and the high-level steps are shown below.

1. Create a custom SAP ABAP structure using SAP Transaction **SE11** for **ZSERVICE\_MESSAGE\_IOT**.

The screenshot shows the SAP SE11 transaction interface for creating a new structure. The 'Structure' field is set to 'ZSERVICE\_MESSAGE\_IOT' and is marked as 'Active'. The 'Short Description' is 'Message'. The 'Components' tab is selected, showing a table of components.

| Component | Typing Method | Component Type | Data Type | Length | Deci... |
|-----------|---------------|----------------|-----------|--------|---------|
| TYPE      | Types         | BAPI_MTYPE     | CHAR      | 1      | 0       |
| MSG       | Types         | BAPI_MSG       | CHAR      | 220    | 0       |

2. Create a custom SAP ABAP structure using SAP Transaction **SE11** for **ZPM\_SERVICE\_NOTIF\_STR**.

The screenshot shows the SAP SE11 transaction interface for creating a new structure. The 'Structure' field is set to 'ZPM\_SERVICE\_NOTIF\_STR' and is marked as 'Active'. The 'Short Description' is 'Structure for Service Notification'. The 'Components' tab is selected, showing a table of components.

| Component  | Typing Method | Component Type   | Data Type | Length | Deci... |
|------------|---------------|------------------|-----------|--------|---------|
| NOTIF_NO   | Types         | QNUM             | CHAR      | 12     | 0       |
| EQUIPMENT  | Types         | EQUNR            | CHAR      | 18     | 0       |
| FUNCT_LOC  | Types         | TPLNR            | CHAR      | 30     | 0       |
| SHORT_TEXT | Types         | QMTXT            | CHAR      | 40     | 0       |
| LONG_TEXT  | Types         | TDLINE           | CHAR      | 132    | 0       |
| MESSAGE    | Types         | ZSERVICE_MESSAGE | CHAR      | 0      | 0       |

3. Create an SAP gateway service in SAP Transaction **SEGW** using the SAP ABAP structure created in the previous step.

The screenshot shows the 'Create an Entity Type or Complex Type' dialog box. The 'Name' field is set to 'NOTIF\_CREATE'. The 'Entity Type' radio button is selected. The 'Import from ABAP Structure' button is visible, and the 'ABAP Structure' field is set to 'ZPM\_SERVICE\_NOTIF'.

4. Redefine the method **NOTIF\_CREATESET\_CREATE\_ENTITY** in the Service Implementation section.

The screenshot shows the SAP Service Implementation section. The method 'NOTIF\_CREATESET\_CREATE\_ENTITY' is selected. A context menu is open, showing options: 'Change', 'Display', and 'Redefine'. The 'Redefine' option is highlighted.

5. Update the ABAP code to call the function module **BAPI\_ALM\_NOTIF\_CREATE**, **BAPI\_ALM\_NOTIF\_SAVE** and **BAPI\_TRANSACTION\_COMMIT**. This part of the code create a service notification when a HTTP POST request is

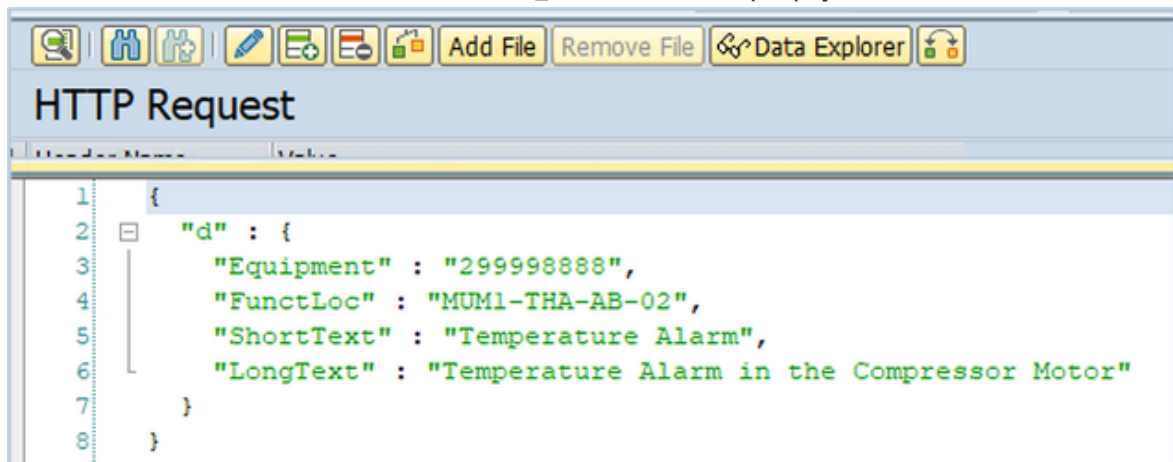


triggered from SAP OData services.

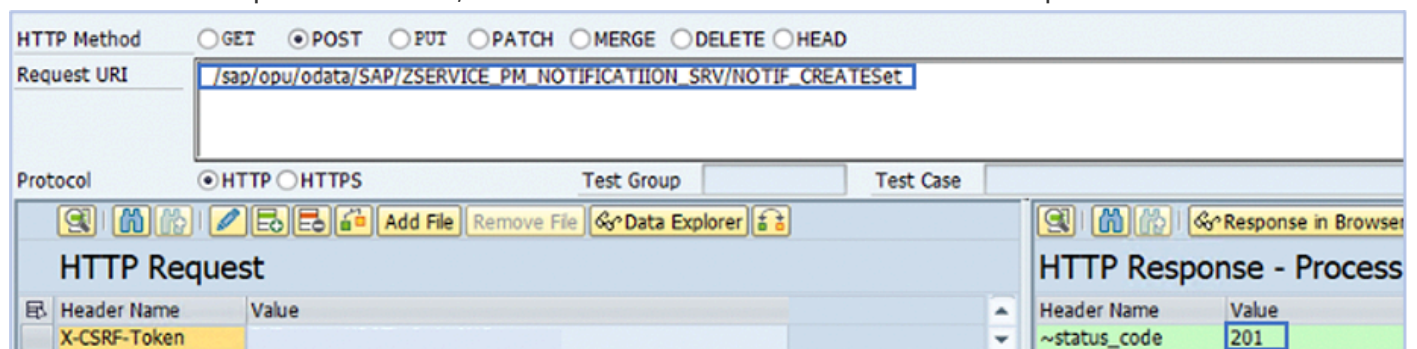
```
"Call the BAPI to create the Notification
CALL FUNCTION 'BAPI_ALM_NOTIF_CREATE'
  EXPORTING
    notif_type           = 'M2'
    notifheader          = lwa_notif_header
  IMPORTING
    notifheader_export   = lwa_notif_export
  TABLES
    longtexts            = lt_longtext
    return               = lt_return
IF lwa_notif_export IS NOT INITIAL.
  "Call BAPI to save the Notification
  CALL FUNCTION 'BAPI_ALM_NOTIF_SAVE'
    EXPORTING
      number             = lwa_notif_export-notif_no
    IMPORTING
      notifheader        = lwa_notif_export2
    TABLES
      return              = lt_return
```

6. Register the OData service in using SAP transaction **/IWFND/MAINT\_SERVICE**

7. Test the OData service in SAP transaction **/IWFND/GW\_CLIENT**. An example payload is shown below.



8. If the HTTP POST request is successful, the returned status code should be 201. An example screenshot is shown below.



## AWS Solution Deployment

AWS Cloud Development Kit (CDK) lets you define your cloud infrastructure as code in one of five supported programming languages. For AWS CDK installation steps, refer to the [AWS CDK documentation](#).

1. Clone the [aws-iot-sap-condition-monitoring-demo](https://github.com/aws-samples/aws-iot-sap-condition-monitoring-demo) repository

Bash

```
git clone https://github.com/aws-samples/aws-iot-sap-condition-monitoring-demo.git
```

2. Create an SAP user for this demo and add the OData service to the menu of back-end PFCG roles. This should include the start authorizations for the OData service in the back-end system and the business authorizations for creating the service notification.
3. Update the cdk.json. An example of the variables are shown below.

| Variable              | Description                                                               | Example                      |
|-----------------------|---------------------------------------------------------------------------|------------------------------|
| thing_name            | Name of the IoT device                                                    | IoT_thing                    |
| Type                  | Name of the device type                                                   | Compressor                   |
| Equipment             | SAP Equipment number                                                      | 299998888                    |
| FunctLoc              | SAP Functional location                                                   | SAPFunction                  |
| temperature_min       | Minimum temperature                                                       | 12                           |
| temperature_max       | Maximum temperature                                                       | 14                           |
| sns_alert_email_topic | Temperature alarm                                                         | TemperatureAlarm             |
| alarm_emails          | Email address                                                             | example@email.address        |
| odpEntitySetName      | ODP entity set name                                                       | NOTIF_CREATESet              |
| odpEntityServiceName  | ODP service name                                                          | ZPM_SERVICE_NOTIFICATION_SRV |
| sapHostName           | SAP host name                                                             | saponawshostname             |
| sapPort               | SAP port                                                                  | 8000                         |
| sapUsername           | SAP user name, CDK stored this in <a href="#">AWS Secrets Manager</a>     | Johndoesap                   |
| sapPassword           | SAP user password, CDK stored this in <a href="#">AWS Secrets Manager</a> | Johndoeapassword             |

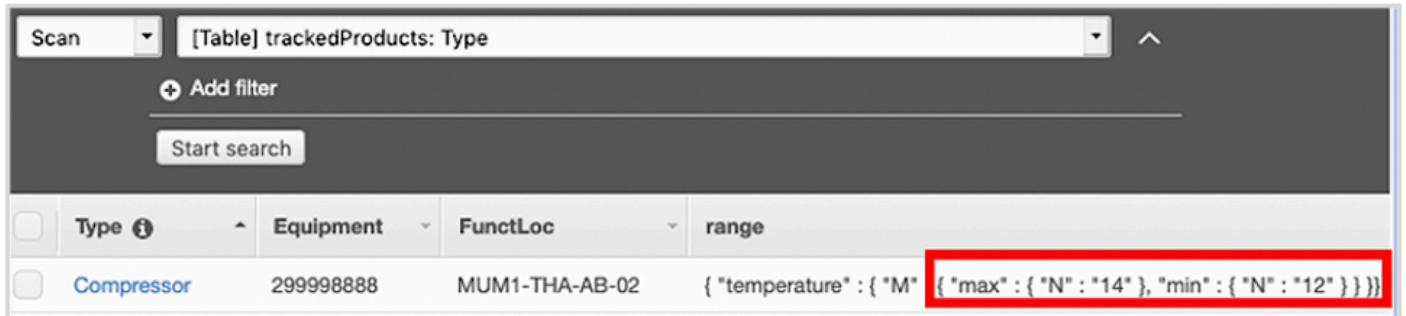
4. Follow the instruction in the [readme section](#) of repository to complete the setup and to run the simulator.
5. Check the stack deployment in AWS CloudFormation. An example of successful deployment is shown below.

| Stacks (22)                                                     |                   |
|-----------------------------------------------------------------|-------------------|
| <input type="text" value="Filter by stack name"/>               |                   |
| Stack name                                                      | Status            |
| <input type="radio"/> <a href="#">cdk-iot-for-sap-analytics</a> | ✔ CREATE_COMPLETE |
| <input type="radio"/> <a href="#">cdk-iot-for-sap-sap</a>       | ✔ CREATE_COMPLETE |
| <input type="radio"/> <a href="#">cdk-iot-for-sap-iot</a>       | ✔ CREATE_COMPLETE |

## Simulate a malfunction of the compressor

Now that we've successfully deployment the solution, let's run a test!

1. The simulator uses the **temperature\_min** and **temperature\_max** variables to simulate temperature of the compressor. These are defined in cdk.json file as part of the CDK deployment. Refer to the [simulator.py script](#) for the code for this simulator.
2. These are stored as an entry in a DynamoDB Table. The minimum temperature (12) and maximum temperature (14) are shown in the DynamoDB table as shown below.



| Type       | Equipment | FunctLoc       | range                                                                          |
|------------|-----------|----------------|--------------------------------------------------------------------------------|
| Compressor | 299998888 | MUM1-THA-AB-02 | { "temperature" : { "M" { "max" : { "N" : "14" }, "min" : { "N" : "12" } } } } |

3. In AWS IoT Analytics, select the DATA SET called **cdksapbloganalyticsdataset**, which was defined in the CDK stack. Then click Run now to view the actual temperate generated. An example is shown below.



| endTime              | durS | thingname | temperature_degC... | humidity_percent... |
|----------------------|------|-----------|---------------------|---------------------|
| 2021-03-11T15:50:... | 63.0 | tracker   | 13.095327904900...  | 12.601903089155...  |

4. You can view the readings generated by the simulator in AWS IoT Events. Go to Detector models, choose **CDKSAPBlogDetectorModel** in detector models, then select tracker in the key value. An example of the temperature in



degree is shown below.

The screenshot shows the AWS IoT Events console. On the left, the 'Detector models' tab is selected. The main area displays the configuration for the 'CDKSAPBlogDetectorModel'. The 'Name' field is 'CDKSAPBlogDetectorModel' and the 'Key value' is 'tracker'. Below this, the 'General information' section shows the 'State' as 'TemplinRange' and the 'Date created' as 'Thu Mar 11 2021'. The 'Variables' section shows a table with one variable: 'temperature\_degC' with a value of '12.930699612693592'.

| Name             | Value              |
|------------------|--------------------|
| temperature_degC | 12.930699612693592 |

- To simulate an anomaly, modify the table temperature values in the DynamoDB table as shown in step 2. As an example, we have changed the maximum from 14 to 13 as shown below.

The screenshot shows the AWS IoT Events console. On the left, the 'Items' tab is selected. The main area displays the configuration for the 'CDKSAPBlogDetectorModel'. The 'Name' field is 'CDKSAPBlogDetectorModel' and the 'Key value' is 'tracker'. Below this, the 'General information' section shows the 'State' as 'TemplinRange' and the 'Date created' as 'Thu Mar 11 2021'. The 'Variables' section shows a table with one variable: 'temperature\_degC' with a value of '12.930699612693592'.

| Name             | Value              |
|------------------|--------------------|
| temperature_degC | 12.930699612693592 |

- Check The readings generated by the simulator in AWS IoT Events and this should now reflect the temperature range set in the DynamoDB.
- If the anomaly continues for more than 5 minutes, an alarm will be triggered to create a service notification in SAP. In addition, an Amazon Simple Notification Services (SNS) notification to the subscriber. An example of email notification is shown below.

The screenshot shows an AWS Notification Message. The subject is 'TemperatureAlarmSap <no-reply@sn>'. The body contains the following text:

Service Notification 000010000436 saved in SAP for temperature alarm

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:

<https://aws.amazon.com/support>

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

8. Validate the service notification in SAP. Go to SAP Transaction **IW23**. Enter the SAP service maintenance notification generated. Click enter to verify the entries.

The screenshot shows the SAP 'Display PM Notification: Malfunction Report' interface. At the top, the title bar reads 'Display PM Notification: Malfunction Report'. Below the title bar, there are several icons. The main data area contains the following fields:

- Notification: 10000436 M2 Temperature Alarm[TempInRange:inRange]
- Notific. Status: OSNO

Below these fields are four tabs: 'Notification', 'Reference object', 'Malfunction Data', and 'Location data'. The 'Reference object' tab is currently selected, showing the following data:

- Functional loc.: MUM1-THA-AB-02
- Equipment: 299998888
- Assembly: (empty field)
- Functional Location 01: Compressor Motor

Below the 'Reference object' tab is the 'Subject' section, which includes:

- Coding: (empty field)
- Description: Temperature Alarm[TempInRange:inRange]
- Event Log: 11.03.2021 16:29:53 UTC (IOT) Temperature Alarm in Compressor Motor

## Clean up

Don't forget to clean up your AWS account to avoid ongoing charges for resources you created in this blog. Simply follow the instructions outlined in the [cleanup section](#) in the Github repository.

## Summary

Machine failures cause adverse impact on manufacturing operational efficiency and the identification of such critical failures pose a challenge in a traditional manufacturing environment. In this blog post, we have shown an example of how to leverage AWS IoT Core, AWS IoT Analytics and SAP to identify equipment anomalies.

There are many more integration scenarios with SAP on AWS. For example, [Amazon Lookout for Equipment](#) can be used to quickly enable predictive maintenance. AWS also makes it easy for you to create your own Machine Learning (ML) models using [Amazon SageMaker](#), such as Long Short-Term Memory (LSTM) neural network. If you have questions or would like to know about SAP on AWS innovations, please [contact the SAP on AWS team](#) or visit [aws.com/sap](https://aws.com/sap) to learn more. Start building on AWS today and have fun!

TAGS: [#saponaws](#)